

rating_window pool mode — exact behavior and why pools stay large

Context: Army Cell 5 `POOL_GROUPING_MODE` toggle (`pipeline_config.py` → `add_pool_means_and_sizes` in `add_cum_oer_metrics_mod_working.py`).

Audience: Charles on AWS / re-entry — why legacy pools looked too big, what `rating_window` was meant to fix, and why Runs 2–3 use `active_at_eval_thru` instead.

What the backward window columns mean (prerequisite)

Before Cell 5 pools, each **panel row** (one officer × one `snpsht_dt`) gets a **backward OER rating period** from the OER merge in `add_cum_oer_metrics_mod_working.py` :

1. `merge_asof` (forward) on (`pid`, `snpsht_dt`) → the **next** OER whose `eval_thru` is on/after the snapshot.
2. If `snpsht_dt < eval_strt` → backward fields are cleared (snapshot is **before** that rating period starts).
3. Stored as `eval_strt_dt_bwd`, `eval_thru_dt_bwd`, plus `snr_rater_bwd`.

So on a given row, those dates mean: “This is the OER cycle the senior rater (SNR) is working in while this snapshot is live.”

That is **not** the same as Army’s field `snr_rater_rates_this_grd` (official “how many on this OER”). The pool-size probe showed official median ~12 vs computed **legacy** median ~23.

Exactly what `rating_window` does in Cell 5

Config: `POOL_GROUPING_MODE = 'rating_window'`

Code path: Same as `legacy` — a pandas `groupby` + transform, not a time-overlap join.

Group key (four columns — all must match exactly):

```
snpsht_dt × snr_rater_bwd × eval_strt_dt_bwd × eval_thru_dt_bwd
```

From `resolve_pool_group_cols()` in `add_cum_oer_metrics_mod_working.py` :

```
if mode == 'rating_window':
    return [snapshot_date_col, rater_col, eval_strt_col, eval_thru_col]
```

For each row with finite `tb_ratio_fwd_snr` :

1. Find **all rows in the entire Cell 5 panel** with the **same four values**.
2. `pool_size` = count of those rows (with leave-one-out (LOO): minus self if `POOL_EXCLUDE_SELF=True`).
3. `pool_tb_ratio_mean` = mean of peers’ TB ratios in that bucket (LOO excludes self).
4. If count < `POOL_MIN_SIZE` (3) → pool mean/size = **NaN**.

There is **no** check that peers’ windows **overlap in calendar time** beyond sharing **identical** start/end stamps. The mode does not ask “who was active on date *T*?” — only “who shares this **exact ID tuple**?”

Toy example (why it splits vs legacy)

Same SNR *S*, same snapshot day **2021-06-15**:

Officer	eval_strt_bwd	eval_thru_bwd
A	2020-10-01	2021-09-30
B	2020-10-01	2021-09-30
C	2021-01-01	2021-12-31

Legacy (`snpsht`, *S*) : pool = {A,B,C} → LOO size **2** (if all on that snapshot).

rating_window:

- A & B → one bucket → LOO size **1** (often **NaN** with `POOL_MIN_SIZE=3`).
- C → alone → size **0** → **NaN**.

So **rating_window shrinks pools** when the same SNR on the same snapshot day has **different OER window stamps** — legacy wrongly lumps them.

Why pools can still look irrationally large

1. Bulk identical windows (main story)

HRC often assigns **the same** (`eval_strt`, `eval_thru`) to a **large batch** of officers on one snapshot. Then:

```
rating_window bucket size ≈ “everyone on that snapshot in that cycle”
```

That can still be **20–40+** officers — not Army’s ~12, because you’re counting **panel rows in a statistical cohort**, not “officers on this one OER form.”

`rating_window` only splits pools when window **stamps differ** on the same `(snpsht, SNR)`. It does **not** ask “who was actually being rated at OER write?”

2. It still keys on `snpsht_dt`

Peers must appear on the **same snapshot date**, not just share SNR + window. That’s stricter than “all officers this SNR ever rated in this cycle,” but **looser** than “active at OER write date.”

The probe’s `alt rating_window key (snr × eval window)` in `army_pool_size_probe.py` **drops** `snpsht_dt` — that’s why it showed median ~4, p95 ~33 (closer to Army). True `rating_window` in code adds `snpsht_dt` back, so pools can be **larger** than that probe line.

3. Groupby counts rows, not “rating events”

Each matching **panel row** counts. Usually ~ one row per officer per snapshot, but the bucket is still “everyone on this snapshot in this window,” not “OER headcount.”

4. Not the same fix as `active_at_eval_thru`

Mode	Mechanism	Typical median (probes)
legacy	<code>snpsht × SNR</code>	~23
rating_window	<code>snpsht × SNR × strt × thru</code> (exact match)	Between legacy and ~4 depending on stamp duplication
active_at_eval_thru	Overlap: peer’s window covers this row’s <code>eval_thru_dt_bwd</code>	~12 (Alex slide / official scale)

`rating_window` is a static bucket refine.
`active_at_eval_thru` is a temporal peer rule.

Side-by-side: `rating_window` vs `active_at_eval_thru`

`rating_window` (groupby)

Peer ⇔ same `snpsht_dt` AND same `snr_rater_bwd`
AND same `eval_strt_dt_bwd` AND same `eval_thru_dt_bwd`

Implementation: `_add_pool_mean_size()` with `group_cols` from `resolve_pool_group_cols()`.

`active_at_eval_thru` (merge + filter)

For row `i`, anchor `T_i = eval_thru_dt_bwd` (this row)

Peer `j` counts iff:
 `snr_rater_bwd[j] = snr_rater_bwd[i]`
 AND `eval_strt_bwd[j] ≤ T_i ≤ eval_thru_bwd[j]`
 AND (optional) `j ≠ i` for L00

Implementation: `_add_pool_mean_size_active_at_anchor()`.

Peers can have **different** `(strt, thru)` as long as their window **covers** your anchor date. Pools track “who was in the SNR’s active set at OER completion” — much closer to why official counts are smaller.

The active-at path deduplicates to one row per `(pid, snr, strt, thru)` before counting peers.

`active_at_snapshot` (same machinery, different anchor)

Same overlap logic as `active_at_eval_thru`, but anchor `T_i = snpsht_dt` instead of `eval_thru_dt_bwd`. Panel-time variant for time-varying Cox rows, not the Alex OER-write story.

Why the project moved past `rating_window`

Design sequence (2025–2026):

- `(snr, eval_thru)` **alone** — better than legacy but still not temporal overlap when windows stagger.
- `rating_window` — adds full window to the key; helps when legacy over-merges **different** windows on one snapshot, but **still over-counts** when many officers share one stamped cycle.
- `active_at_eval_thru` — overlap at `eval_thru_dt_bwd`; probe medians landed near Army official scale.

`rating_window` was a **stepping stone**, not the final Alex-facing grain. Sensitivity matrix Runs 2–3 use `active_at_eval_thru`, not `rating_window`.

If you experiment with `rating_window` on AWS

- Set `POOL_GROUPING_MODE = 'rating_window'` in `pipeline_config.py`.
- Re-run Cell 5 → 11 (feather must rebuild).
- Run:

```
python -u talent/re_entry/army_pool_size_probe.py
```

Read:

- `Cell 5 pool_size_snr_fwd (all rows, fresh)` — actual mode output.
- `alt rating_window key (snr x eval window)` — **sanity line only** (no `snpsht_dt`; not identical to true mode).

Expect: **smaller than legacy** when many window stamps exist per snapshot; **still often too big** vs official ~12 if big batches share one stamp.

Config reference (`pipeline_config.py`)

```
# legacy          - snpsht_dt x snr_rater_bwd (original pipeline)
# rating_window   - + eval_strt_dt_bwd, eval_thru_dt_bwd on same snapshot
# active_at_eval_thru - peers SNR-rating at eval_thru_dt_bwd (Alex slide / OER write time)
# active_at_snapshot - peers SNR-rating at snpsht_dt (panel-time variant)
POOL_GROUPING_MODE = 'active_at_eval_thru' # current matrix default for thru runs
POOL_ANCHOR_COL = 'eval_thru_dt_bwd'
POOL_EVAL_STRT_COL = 'eval_strt_dt_bwd'
POOL_EVAL_THRU_COL = 'eval_thru_dt_bwd'
```

Bottom line

`rating_window` = “Pool me with everyone on **this snapshot day**, with **this SNR**, in **this exact OER period record** (same start **and** end dates).”
It is **exact-match groupby**, not overlap-at-anchor. That is why it **does not fully fix** irrational pool sizes: Army’s ~12 is closer to “**peers active when the OER closes**” — which is `active_at_eval_thru`.

Created 2026-09-23 — from COMPASS thread on Army pool sensitivity / Charles AWS matrix.